

PCAP™ Certified Associate Python Programmer

Complete Study Guide — Exam PCAP-31-03

40 Questions

70% Pass Mark

65 Minutes

5 Sections

Knowledge areas

Study Priority Overview

Priority	Section	Weight	Questions
■ Must Master	4 – Object-Oriented Programming	34%	12
■ High Priority	5 – Miscellaneous (Lambdas, Closures, I/O)	22%	9
■ Important	3 – Strings	18%	8
■ Don't Skip	2 – Exceptions	14%	5
■ Lighter	1 – Modules & Packages	12%	6

Section 1: Modules & Packages

12% · 6 questions

■ Import Variants

- `import module_name`
- `from module import name`
- `import module as alias`
- `from module import *`
- Advanced qualifying for nested modules
- `dir()` function to inspect module contents
- `sys.path` variable — how Python finds modules

■ Standard Library Modules

- `math`: `ceil()`, `floor()`, `trunc()`, `factorial()`, `hypot()`, `sqrt()`
- `random`: `random()`, `seed()`, `choice()`, `sample()`
- `platform`: `platform()`, `machine()`, `processor()`, `system()`, `version()`, `python_implementation()`, `python_version_tuple()`

■ User-Defined Modules & Packages

- `__name__` variable and the `if __name__ == '__main__':` pattern
- `__init__.py` — making a directory a package
- `__pycache__` directory — compiled bytecode
- Public vs. private variables in modules
- Nested packages vs. directory trees
- Searching for/through modules and packages

Section 2: Exceptions

14% · 5 questions

■ Exception Handling Syntax

- try / except blocks
- Ordering except branches (most specific first)
- except ExceptionType as variable
- Grouped exceptions: except (TypeError, ValueError)
- else clause — runs when no exception was raised
- finally clause — always runs

■ Raising & Asserting

- raise — re-raise current exception
- raise ExceptionType(message) — raise new exception
- assert condition, message — debugging assertions

■ Exception Hierarchy & Classes

- Built-in hierarchy: BaseException → Exception → ...
- Exception classes: args property
- Common built-ins: ValueError, TypeError, IndexError, KeyError, IOError, etc.

■ Custom Exceptions

- class MyError(Exception): pass
- Overriding __init__ and __str__ for custom messages
- Integrating custom exceptions into existing hierarchies

Section 3: Strings

18% · 8 questions

■ Character Encoding

- ASCII — 7-bit, 128 characters
- Unicode — universal character set (code points)
- UTF-8 — variable-length encoding of Unicode
- Escape sequences: \n, \t, \\\, \', \", \uXXXX
- ord(char) → integer code point
- chr(int) → character

■ String Operations

- Indexing: s[0], s[-1]
- Slicing: s[start:stop:step]
- Immutability — strings cannot be changed in place
- Iteration: for char in string
- Concatenation: + operator
- Repetition: * operator
- Comparisons: ==, !=, <, >, <=, >= (lexicographic)
- Membership: in, not in

■ Essential String Methods

- .upper(), .lower(), .capitalize(), .title()
- .isalpha(), .isdigit(), .isalnum(), .isspace(), .isupper(), .islower()
- .strip(), .lstrip(), .rstrip()

- `.split(sep)`, `.join(iterable)`
- `.find(sub)`, `.rfind(sub)` — return index or -1
- `.index(sub)` — like `find()` but raises `ValueError` if not found
- `.replace(old, new)`
- `.startswith(prefix)`, `.endswith(suffix)`
- `.sorted()` — returns a sorted list of characters

Section 4: Object-Oriented Programming

34% · 12 questions

■ Core OOP Concepts

- Class — blueprint for creating objects
- Object / Instance — a concrete realisation of a class
- Property / Attribute — data stored in an object
- Method — function defined inside a class
- Encapsulation — bundling data and behaviour
- Inheritance — deriving a new class from an existing one
- Superclass / Subclass (parent / child class)

■ Class & Instance Variables

- Instance variables: defined in `__init__`, unique per object
- Class variables: defined in class body, shared across all instances
- `__dict__` property — inspect attributes of objects and classes
- Private components: `_name` (convention) and `__name` (name mangling)
- Name mangling: `__attr` becomes `_ClassName__attr`

■ Methods & Constructors

- Defining methods inside a class
- The `self` parameter — always the first argument
- `__init__(self, ...)` — constructor, called on object creation
- Calling methods: `obj.method()`

■ Introspection

- `hasattr(obj, 'attr')` — check if attribute exists
- `getattr` / `setattr` / `delattr`
- `__name__` — class name as string
- `__module__` — module where class is defined
- `__bases__` — tuple of parent classes

■ Inheritance & Polymorphism

- Single inheritance: `class Child(Parent):`
- Multiple inheritance: `class Child(Parent1, Parent2):`
- `isinstance(obj, Class)` — check type
- `is` / `is not` — identity operators
- Method overriding — redefine parent method in child
- Calling parent method: `super().method()`
- Polymorphism — same method name, different behaviour
- Overriding `__str__()` — controls `str()` and `print()` output
- Diamond problem & MRO (Method Resolution Order)

Section 5: Miscellaneous (Lambdas, Closures, I/O)

22% · 9 questions

■ List Comprehensions

- Basic: [expression for item in iterable]
- With condition: [expr for item in iterable if condition]
- Nested: [expr for x in outer for y in inner]
- Dictionary comprehension: {k: v for k, v in pairs}

■ Lambda Functions

- Syntax: lambda args: expression
- Anonymous, single-expression functions
- Passing lambdas as arguments to other functions
- map(func, iterable) — apply function to each element
- filter(func, iterable) — keep elements where func returns True
- Combining map/filter with lambdas

■ Closures

- A function that captures variables from its enclosing scope
- The enclosing scope must have already returned
- Why closures? — factory functions, callbacks, data hiding
- nonlocal keyword — modify enclosing variable

■ File I/O — Concepts

- I/O modes: 'r' read, 'w' write, 'a' append, 'rb'/'wb' binary
- Predefined streams: sys.stdin, sys.stdout, sys.stderr
- File handles vs. streams
- Text mode vs. binary mode differences

■ File I/O — Operations

- open(path, mode) — returns a file object
- errno module — error codes for OS errors
- file.read(n) — read n bytes / all
- file.readline() — read one line
- file.readlines() — read all lines into a list
- file.write(string) — write string
- file.close() — release the file
- with open(...) as f: — context manager (auto-close)
- bytearray — mutable buffer for binary I/O

■ Top Study Tips

- ✓ Focus 40%+ of your time on Section 4 (OOP) — it's the largest section.
- ✓ Use the free Python Essentials 2 course at edube.org — officially aligned with PCAP-31-03.
- ✓ Practice writing classes from scratch: constructors, inheritance, overriding `__str__()`.
- ✓ Learn the exception hierarchy by heart; know which exceptions inherit from which.
- ✓ For strings, memorise the key methods — especially `find()` vs. `index()` and `join()` vs. `split()`.
- ✓ Practise list comprehensions and `lambda+map+filter` combos with real code.
- ✓ Do at least 2 full timed practice tests (40 questions in 65 minutes) before exam day.

Source: Python Institute Official Exam Syllabus (PCAP-31-03) · pythoninstitute.org/pcap-exam-syllabus